Task 9 – Record All Tools & Documents

Student: Prince Raj

Practical Record

This document consolidates the tools, configuration files, commands, screenshots, and observations from the completed Docker and Kubernetes practical work.

Area	Tools / Documents Recorded
Docker	Docker Desktop, Docker CLI, Dockerfile, Docker Compose, Docker images, containers, volumes, networks
Application	Python, Flask, requirements.txt, application container
Kubernetes	Docker Desktop Kubernetes, kubectl, Deployment YAML, Service YAML, NGINX
CI/CD	Jenkins Pipeline, kubectl, Kubernetes Deployment and Service manifests
Environment	Windows PowerShell / Command Prompt, local Docker Desktop Kubernetes cluster

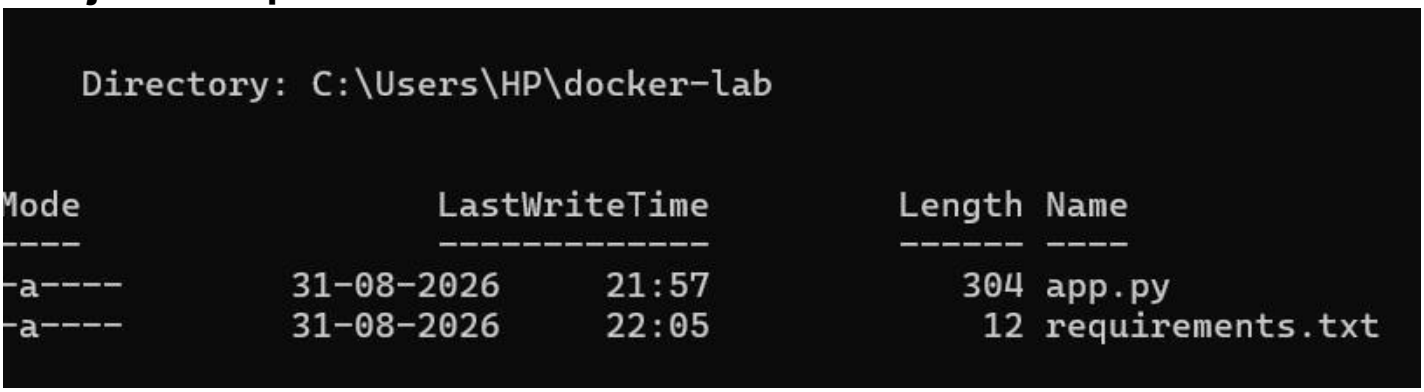
Recorded Practical Areas

Docker containerization and lifecycle management; Docker storage and networking; Kubernetes deployment, services, scaling, rolling update and rollback; containerization and orchestration reporting; and Jenkins-based continuous deployment.

Docker Task 1 – Containerization

The practical recorded the Docker project structure and the local Docker workflow for the Flask application.

Project Setup



```
    Directory: C:\Users\HP\docker-lab


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        31-08-2026     21:57            304 app.py
-a----        31-08-2026     22:05             12 requirements.txt
```

Figure 1. Docker project files shown in the practical.

Key Docker record

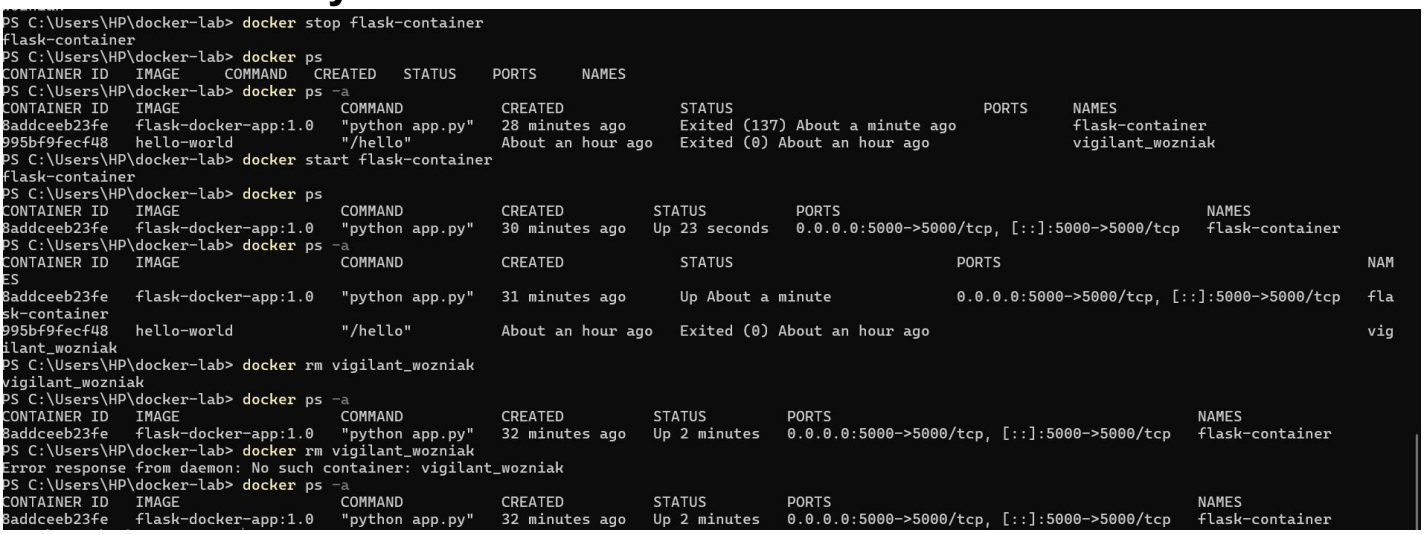
Dockerfile: Python 3.11-slim base image
Application port: 5000

Main workflow: Dockerfile → Image → Container → Running Flask application

Docker Task 2 – Containers, Images & Compose

The practical documented container stop/start/remove operations, Docker image handling, Docker Hub push, and Docker Compose configuration.

Container Lifecycle



```
PS C:\Users\HP\docker-lab> docker stop flask-container
flask-container
PS C:\Users\HP\docker-lab> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
PS C:\Users\HP\docker-lab> docker ps -a
CONTAINER ID   IMAGE                 COMMAND           CREATED             STATUS                          PORTS     NAMES
8addceeb23fe   flask-docker-app:1.0   "python app.py"   28 minutes ago      Exited (137) About a minute ago             flask-container
995bf9fecf48   hello-world           "/hello"          About an hour ago   Exited (0) About an hour ago                vigilant_wozniak
PS C:\Users\HP\docker-lab> docker start flask-container
flask-container
PS C:\Users\HP\docker-lab> docker ps
CONTAINER ID   IMAGE                 COMMAND           CREATED          STATUS          PORTS                                       NAMES
8addceeb23fe   flask-docker-app:1.0   "python app.py"   30 minutes ago   Up 23 seconds   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   flask-container
PS C:\Users\HP\docker-lab> docker ps -a
CONTAINER ID   IMAGE                 COMMAND           CREATED             STATUS                         PORTS                                       NAM
ES
8addceeb23fe   flask-docker-app:1.0   "python app.py"   31 minutes ago      Up About a minute              0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   fla
sk-container
995bf9fecf48   hello-world           "/hello"          About an hour ago   Exited (0) About an hour ago                                                vig
ilant_wozniak
PS C:\Users\HP\docker-lab> docker rm vigilant_wozniak
vigilant_wozniak
PS C:\Users\HP\docker-lab> docker ps -a
CONTAINER ID   IMAGE                 COMMAND           CREATED          STATUS          PORTS                                       NAMES
8addceeb23fe   flask-docker-app:1.0   "python app.py"   32 minutes ago   Up 2 minutes    0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   flask-container
PS C:\Users\HP\docker-lab> docker rm vigilant_wozniak
Error response from daemon: No such container: vigilant_wozniak
PS C:\Users\HP\docker-lab> docker ps -a
CONTAINER ID   IMAGE                 COMMAND           CREATED          STATUS          PORTS                                       NAMES
8addceeb23fe   flask-docker-app:1.0   "python app.py"   32 minutes ago   Up 2 minutes    0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   flask-container
```

Figure 3. Docker container stop, start, listing and removal commands.

```
PS C:\Users\HP\docker-lab> docker push anand4123/flask-docker-app:1.0
The push refers to repository [docker.io/anand4123/flask-docker-app]
87e1b7cce023: Pushed
6310eb16bf42: Pushed
7ea3ea474f31: Pushed
44136fa355b3: Pushed
c86306e32cd0: Pushed
a14578096eda: Pushed
7eef5e4f9f95: Pushed
59cfb10edb51: Pushed
23ecc929b5d5: Pushed
0318cf563730: Pushed
1.0: digest: sha256:c97ce1599a3bbefa00fb3cbec2779f39941f6d002609359e1d8e0f0509ada2a0 size: 856
```

Figure 4. Docker image pushed to the Docker Hub repository.

```
PS C:\Users\HP\docker-lab> docker compose config
name: docker-lab
services:
  redis:
    container_name: flask-redis
    image: redis:7-alpine
    networks:
      default: null
  web:
    build:
      context: C:\Users\HP\docker-lab
      dockerfile: Dockerfile
    container_name: flask-web
    depends_on:
      redis:
        condition: service_started
        required: true
    networks:
      default: null
    ports:
      - mode: ingress
        target: 5000
        published: "5000"
        protocol: tcp
networks:
  default:
    name: docker-lab_default
```

Figure 5. Docker Compose configuration for the Flask web service and Redis.

Docker Task 3 – Storage

The storage practical recorded container-layer data behavior, named-volume storage, and bind-mount storage.

Container Layer and Bind Mount Evidence

```
PS C:\Users\HP> docker run -it --name storage-test ubuntu bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
f3db1cd94078: Pull complete
06e9d71331fb: Pull complete
f79afc4bbefe: Download complete
Digest: sha256:2260313b31c8c011cd2eebe728008efac1b3982be73eb71348ea2648d2c0e09b
Status: Downloaded newer image for ubuntu:latest
root@8197e0c185d4:/#
```

Figure 6. Ubuntu container created for the storage experiment.

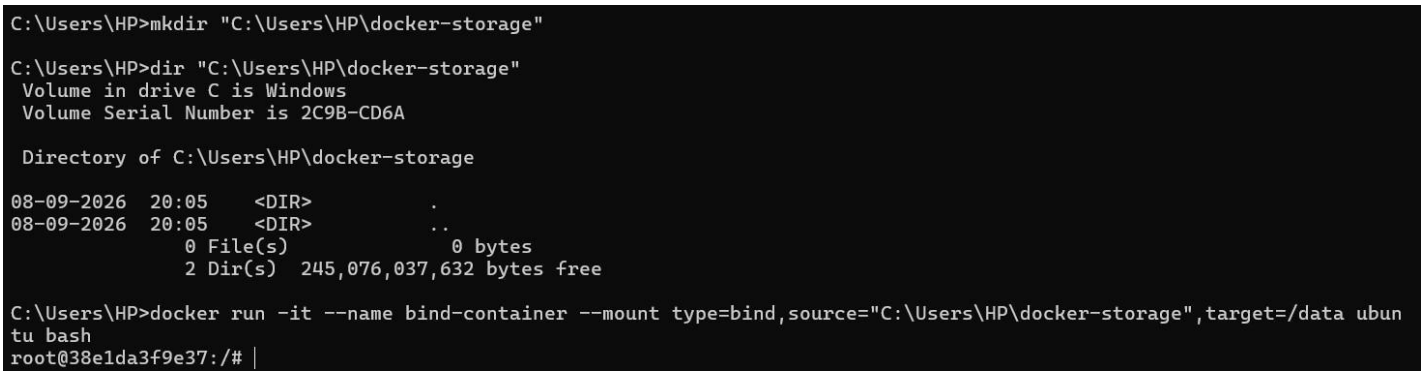


Figure 7. Correct Windows bind-mount path used to start the container.

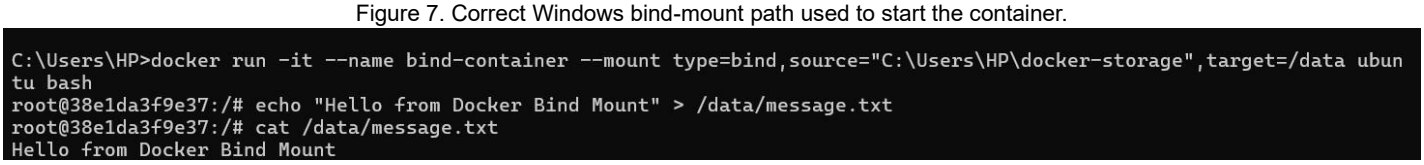


Figure 8. Data written to /data and read back from the bind mount.

Docker Task 4 – Network

The network practical documented a custom Docker network, a web-server container, a client container, container-to-container communication, and network inspection.

Network Communication

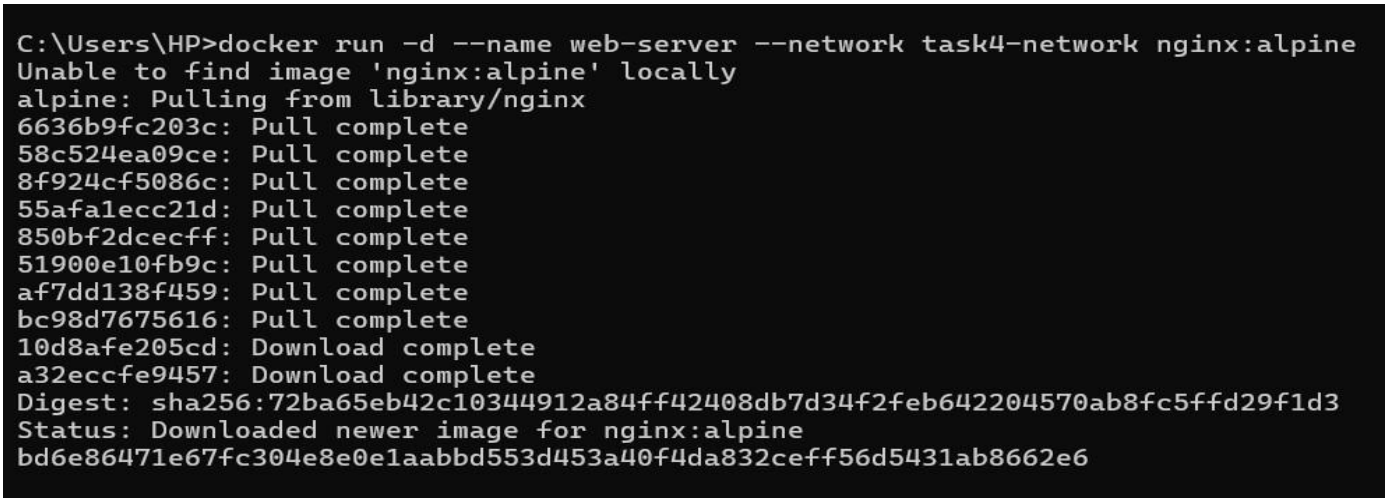


Figure 9. NGINX web-server container created on the custom Docker network.

```
/ # wget -qO- http://web-server
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Figure 10. Client container retrieved the NGINX page using the container name.


```
C:\Users\HP>docker network inspect task4-network
[
    {
        "Name": "task4-network",
        "Id": "7c32d3dda81cb35c7c796d0279add1c0ecf22ab9e3c994bbfd955b35ba57f02a",
        "Created": "2026-09-08T14:47:01.4426489837",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.19.0.0/16",
                    "Gateway": "172.19.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "com.docker.network.enable_ipv4": "true",
            "com.docker.network.enable_ipv6": "false"
        },
        "Labels": {},
        "Containers": {
            "bd6e86471e67fc304e8e0e1aabbd553d453a40f4da832ceff56d5431ab8662e6": {
                "Name": "web-server",
                "EndpointID": "f8d5e84f04acf79b24f4996f9d969daa5e2ab8ae569620ae1f35ce356ab31883",
                "MacAddress": "de:6d:4e:bc:b1:e2",
                "IPv4Address": "172.19.0.2/16",
                "IPv6Address": ""
            }
        },
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.19.0.0/16": {
                        "IPsInUse": 4,
                        "DynamicIPsAvailable": 65532
                    }
                }
            }
        }
    }
]
```


Figure 11. Docker network inspection showing the connected web-server container.

Kubernetes Task 1 – Deploy Application

The Kubernetes practical recorded the cluster status, Service and endpoint configuration, and successful NGINX application access.

Service and Application Verification

```
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP>kubectl get nodes
NAME                   STATUS   ROLES           AGE    VERSION
desktop-control-plane   Ready    control-plane   2m7s   v1.36.1

C:\Users\HP>kubectl get pods
No resources found in default namespace.
```

Figure 12. Kubernetes node status and initial Pod state.

```
C:\Users\HP\kubernetes-task-1>kubectl get services
NAME                 TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
kubernetes           ClusterIP   10.96.0.1      <none>        443/TCP        40m
nginx-demo-service   NodePort    10.96.168.87   <none>        80:31450/TCP   10s

C:\Users\HP\kubernetes-task-1>kubectl get endpoints nginx-demo-service
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME                 ENDPOINTS       AGE
nginx-demo-service   10.244.0.5:80   21s
```

Figure 13. NodePort Service and endpoint verification.



Figure 14. NGINX application successfully displayed in the browser.

Kubernetes Task 2 – Deployment Strategies

The practical demonstrated scaling to three replicas, a rolling image update, and deployment rollback.

Scaling

```
PS C:\Users\HP> cd $env:USERPROFILE\kubernetes-task-1
PS C:\Users\HP\kubernetes-task-1> dir


    Directory: C:\Users\HP\kubernetes-task-1


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         22-09-2026     20:14            347 deployment.yaml
-a----         22-09-2026     20:27            196 service.yaml


PS C:\Users\HP\kubernetes-task-1> kubectl get deployment
NAME         READY   UP-TO-DATE   AVAILABLE   AGE
nginx-demo   1/1     1            1           35m
PS C:\Users\HP\kubernetes-task-1> kubectl scale deployment nginx-demo --replicas=3
deployment.apps/nginx-demo scaled
PS C:\Users\HP\kubernetes-task-1> kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
nginx-demo-6f5d9c55b8-74wjz   1/1     Running   0          36m
nginx-demo-6f5d9c55b8-g6rlp   1/1     Running   0          25s
nginx-demo-6f5d9c55b8-kzfgr   1/1     Running   0          25s
```

Figure 15. Deployment scaled to three running replicas.

Rolling Update

```
PS C:\Users\HP\kubernetes-task-1> kubectl rollout history deployment/nginx-demo
deployment.apps/nginx-demo
REVISION  CHANGE-CAUSE
1         <none>

PS C:\Users\HP\kubernetes-task-1> kubectl set image deployment/nginx-demo nginx=nginx:alpine
deployment.apps/nginx-demo image updated
PS C:\Users\HP\kubernetes-task-1> kubectl rollout status deployment/nginx-demo
deployment "nginx-demo" successfully rolled out
PS C:\Users\HP\kubernetes-task-1> kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
nginx-demo-7bc974fb64-jc6xs   1/1     Running   0          33s
nginx-demo-7bc974fb64-kcq76   1/1     Running   0          29s
nginx-demo-7bc974fb64-n66jj   1/1     Running   0          28s
```

Figure 16. NGINX image updated to nginx:alpine and rollout completed successfully.

Rollback

```
PS C:\Users\HP\kubernetes-task-1> kubectl rollout undo deployment/nginx-demo
Warning: resource deployments/nginx-demo was previously managed with 'kubectl apply'. Rolling back will not update the k
ubectl.kubernetes.io/last-applied-configuration annotation, which may cause unexpected behavior on future 'kubectl apply
' operations. Consider using 'kubectl apply' with your previous configuration file instead.
deployment.apps/nginx-demo rolled back
```

Figure 17. Deployment rollback command completed successfully.

Kubernetes Task 3 – Containerization & Orchestration Report

The orchestration report consolidated Deployment and Service behavior, Pod verification, scaling, rolling update, rollback, endpoint verification and browser access.

Recorded Kubernetes Manifest Details

```
Deployment: nginx-demo
Container image: nginx:latest
Container port: 80
Service: nginx-demo-service
Service type: NodePort
Target port: 80
Access: localhost:8081 through kubectl port-forward
```

Observations

The Kubernetes control-plane node was Ready; the Deployment created and maintained the NGINX Pod; the Service endpoint connected the Service to the Pod; scaling increased the application to three replicas; the rolling update completed successfully; and rollback returned the Deployment to the previous image configuration.

Supporting Evidence

```
C:\Users\HP\kubernetes-task-1>kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
nginx-demo-6f5d9c55b8-74wjz   1/1     Running   0          2m3s
```

Figure 18. Running NGINX Pod verification from the orchestration report.

```
PS C:\Users\HP\kubernetes-task-1> kubectl rollout undo deployment/nginx-demo
Warning: resource deployments/nginx-demo was previously managed with 'kubectl apply'. Rolling back will not update the k
ubectl.kubernetes.io/last-applied-configuration annotation, which may cause unexpected behavior on future 'kubectl apply
' operations. Consider using 'kubectl apply' with your previous configuration file instead.
deployment.apps/nginx-demo rolled back
```

Figure 19. Rollback evidence recorded in the orchestration report.

Task 8 – Continuous Deployment

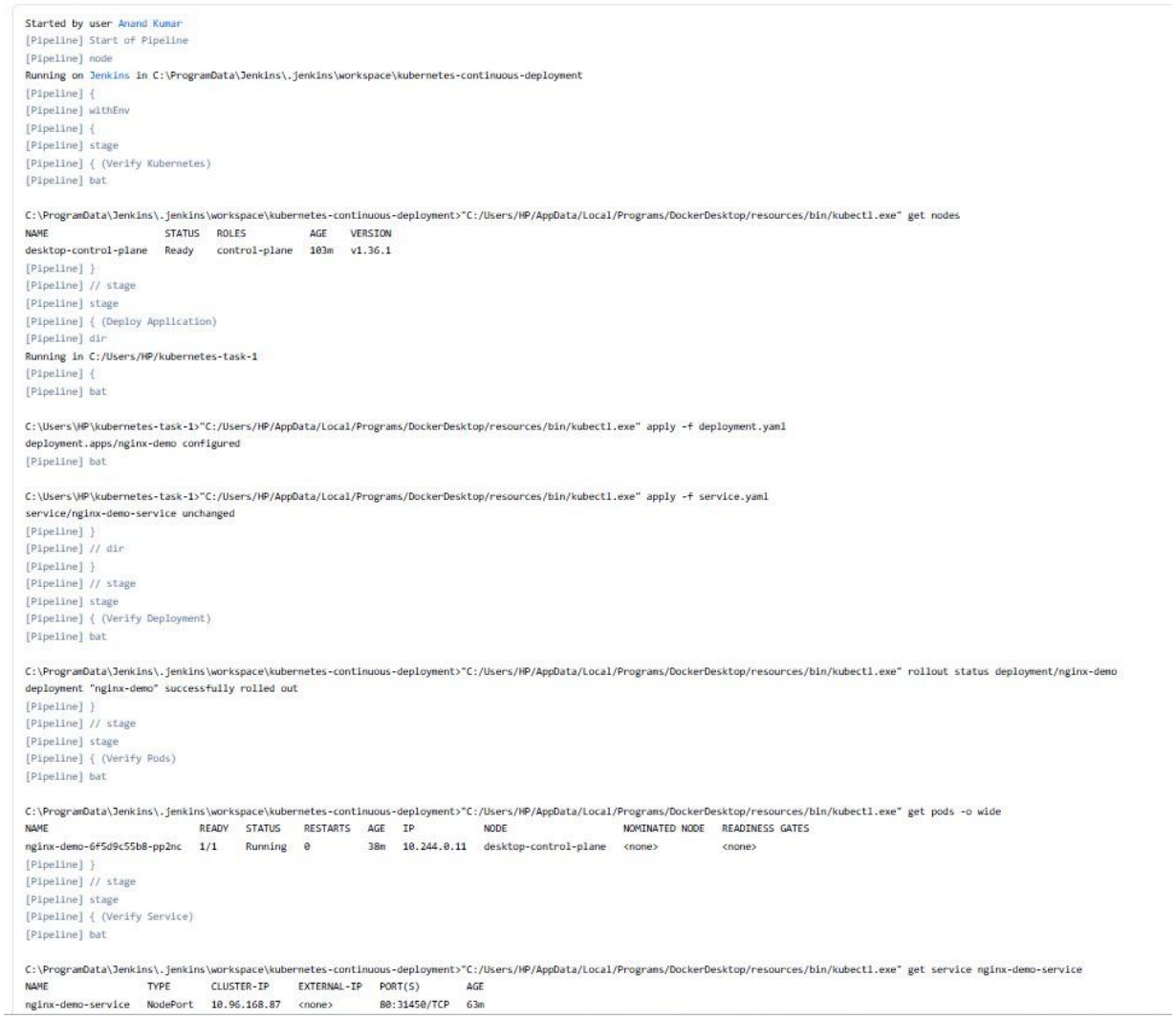
Jenkins was configured with a Kubernetes continuous-deployment pipeline. The successful build applied the Deployment and Service manifests and verified the Kubernetes resources.

Successful Jenkins Build



Figure 20. Jenkins kubernetes-continuous-deployment Build #2 completed successfully.

Jenkins Console Output



```
Started by user Anand Kumar
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\kubernetes-continuous-deployment
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Verify Kubernetes)
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\kubernetes-continuous-deployment>"C:/Users/HP/AppData/Local/Programs/DockerDesktop/resources/bin/kubectl.exe" get nodes
NAME                 STATUS   ROLES           AGE    VERSION
desktop-control-plane   Ready    control-plane   103m   v1.36.1
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy Application)
[Pipeline] dir
Running in C:/Users/HP/kubernetes-task-1
[Pipeline] {
[Pipeline] bat

C:\Users\HP\kubernetes-task-1>"C:/Users/HP/AppData/Local/Programs/DockerDesktop/resources/bin/kubectl.exe" apply -f deployment.yaml
deployment.apps/nginx-demo configured
[Pipeline] bat

C:\Users\HP\kubernetes-task-1>"C:/Users/HP/AppData/Local/Programs/DockerDesktop/resources/bin/kubectl.exe" apply -f service.yaml
service/nginx-demo-service unchanged
[Pipeline] }
[Pipeline] // dir
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Verify Deployment)
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\kubernetes-continuous-deployment>"C:/Users/HP/AppData/Local/Programs/DockerDesktop/resources/bin/kubectl.exe" rollout status deployment/nginx-demo
deployment "nginx-demo" successfully rolled out
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Verify Pods)
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\kubernetes-continuous-deployment>"C:/Users/HP/AppData/Local/Programs/DockerDesktop/resources/bin/kubectl.exe" get pods -o wide
NAME                          READY   STATUS    RESTARTS   AGE   IP           NODE                 NOMINATED NODE   READINESS GATES
nginx-demo-6f5d9c55b8-pp2nc   1/1     Running   0          38m   10.244.0.11   desktop-control-plane   <none>           <none>
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Verify Service)
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\kubernetes-continuous-deployment>"C:/Users/HP/AppData/Local/Programs/DockerDesktop/resources/bin/kubectl.exe" get service nginx-demo-service
NAME                 TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
nginx-demo-service   NodePort   10.96.168.87   <none>        80:31450/TCP   63m
```

Figure 21. Jenkins console showing kubectl node verification, manifest application, rollout, Pod and Service verification.

Commands & Configuration Record

Important commands recorded across the practical work:

```
Docker:
docker build -t flask-docker-app:1.0 .
docker run -d --name flask-container -p 5000:5000 flask-docker-
app:1.0 docker ps / docker ps -a docker stop flask-container docker
start flask-container docker push anand4123/flask-docker-app:1.0
docker compose config docker volume create app-data docker network
inspect task4-network

Kubernetes:
kubectl get nodes kubectl apply -f deployment.yaml
kubectl apply -f service.yaml kubectl get pods kubectl
get services kubectl get endpoints nginx-demo-service
kubectl scale deployment nginx-demo --replicas=3 kubectl
set image deployment/nginx-demo nginx=nginx:alpine
kubectl rollout status deployment/nginx-demo kubectl
rollout undo deployment/nginx-demo
kubectl port-forward service/nginx-demo-service 8081:80

Jenkins:
kubectl apply -f deployment.yaml kubectl
apply -f service.yaml kubectl rollout
status deployment/nginx-demo kubectl get
pods -o wide
kubectl get service nginx-demo-service
```

Final Record

The practical record contains the Docker and Kubernetes configuration documents, command history, deployment evidence, storage and networking experiments, scaling and rollback results, and the successful Jenkins continuous-deployment execution.